OmniFlow: Adaptive Replay System for Mobile GUI Agents

Abstract

abs Modern mobile UIs change between users, devices, and sessions: layouts re-flow, stable selectors are often absent, and content is personalized. LLM-based GUI agents handle this dynamism by re-reasoning at every step—effective on open-ended tasks, but expensive on the recurring routines that dominate real device use. Each step in their reason-act loop may require a fresh model decision, incurring on-device latency, memory, power, and thermal cost, or expanding the privacy boundary when screens are shipped to a cloud planner. Trajectory reuse is the natural serving target, but existing record-and-replay, memory-retrieval, and program-guided systems still fall back to online repair the moment those same dynamic conditions invalidate a stored action.

We present **OmniFlow**, a plug-in cache for mobile GUI agents whose central design is to *check and remap every recalled action against the live screen before it executes*, so historical executions become reusable Functions without requiring identical layouts, selectors, or coordinates at reuse time. **Omni** writes successful run logs into a Function Store whose assets are API-like `Functions`; the store induces sparse UDEG nodes as a privacy-safe state index that keeps raw XML, screenshots, and personal text outside the shared cache boundary. Recall returns compact executable candidates rather than full trajectories. **Flow** drives a recalled Function on the live device through an observe-check-transfer-act loop that checks each action against the current state, remaps the target, executes when safe, and falls back otherwise. Matched Functions therefore skip online reasoning entirely, while uncertain states remain under the host agent through a structured handoff.

On androidworld benchmarks, OmniFlow enables 2.3× reuse of successful trajectories, improving task success rates by 5.4% and reducing latency by 25% on average, with up to 40% improvement on challenging dynamic tasks. User studies show that OmniFlow's reuse leads to smoother interactions and higher satisfaction, especially on tasks with dynamic content. By enabling robust trajectory reuse, OmniFlow significantly enhances the efficiency and user experience of LLM-based GUI agents in the face of mobile UI dynamism.

1 Introduction

Mobile GUI agents powered by LLMs and VLMs are commonly designed for open-ended task execution. At each step, the agent observes the current screen, invokes a model to decide the next action, executes one GUI operation, and repeats this loop until the task is complete. This design is effective for novel tasks because the agent can reason over arbitrary screens and user goals. However, it is an inefficient serving path for a large class of real mobile workloads: repeated GUI execution.

In daily mobile use, many tasks recur with similar structure. Users repeatedly open the same app pages, search for similar content, fill similar forms, change common settings, confirm routine operations, and navigate through familiar workflows. For these recurring tasks, step-wise reasoning forces every user and every repetition to solve nearly the same interaction sequence again. This repeated model invocation introduces latency, computation, energy, memory, and thermal overhead on edge devices. When the model is served in the cloud, it can also expose screenshots, view hierarchies, and private app content to an external planner. These costs motivate a different serving target for mobile GUI agents: successful executions should be reused whenever the current task and live GUI state are compatible.

Existing systems only partially address this target. Step-wise GUI agents improve action generation or verification, but still invoke a model for each interaction. Per-step acceleration methods reduce the cost of each model call, while retaining the same observe-reason-act loop. RPA-style replay and script-based systems avoid model calls, but bind execution to a fixed trajectory, coordinate sequence, selector set, or app state. Memory-based and graph-based agents retrieve prior task knowledge or UI transition graphs, but the retrieved memory often requires online reinterpretation and provides limited support for execution under dynamic layouts, devices, users, and app versions. As a result, existing approaches mismatch the repeated-execution workload: they either remain too expensive by re-planning every step, or become too brittle by replaying stored actions without sufficient adaptation.

We revisit the objective of repeated mobile GUI execution. For recurring tasks, the system does not need to reproduce an entire historical trajectory, nor does it need to re-invoke the VLM for every step. Instead, it should preserve and re-execute the task effect of reusable interaction fragments. For example, a prior execution that opens a search page, enters a query, selects a result, or confirms a form should become a callable operation that can be reused in later tasks. This objective requires adaptive replay: the system must decide when a historical operation is applicable, transfer it to the current GUI state, execute it safely, and fall back to the underlying GUI agent when reuse is uncertain.

Adaptive replay is difficult because mobile GUI execution is dynamic at two levels. First, repeated tasks exhibit workflow-level dynamics. A live task may diverge from a

recorded path because of login state, permission dialogs, advertisements, pop-ups, loading delays, personalized content, or alternative app entry points. A complete recorded trace may no longer match the live workflow, even though many of its local fragments remain useful. Second, mobile apps exhibit GUI-state-level dynamics. Even when the current screen corresponds to a memorized state, the concrete action target may shift because of device size, density, font scale, orientation, app version, language, user content, or missing accessibility attributes. Thus, a useful replay system must support both partial workflow reuse and state-aware action transfer.

OmniFlow builds on two cooperating components. The first component, Omni representation layer, organizes past executions into reusable execution fragments. A successful run is decomposed into compact state descriptions and callable Functions, where each Function captures a task-relevant operation, such as opening a search page, entering a query, selecting an item, or confirming a form. Each Function is associated with the state condition under which it can be applied, the arguments that should be rebound at runtime, and the action sequence needed to produce the intended task effect. This representation gives the system a finer-grained reuse unit than a full historical trajectory. During a live task, Omni representation layer can recall a Function from any compatible intermediate state, execute the reusable fragment, and return control to the underlying GUI agent when the current workflow moves outside the recalled context. If the execution later reaches another compatible state, reuse can resume from that point. In this way, the memory layer makes replay less dependent on the live task following the same end-to-end path as the original execution.

The second component, Flow execution layer, grounds each recalled Function in the current GUI before execution. A recalled Function describes what operation was useful in the past, while the live screen determines whether and how that operation can be executed now. Therefore, it runs each cached action through an observe-check-transfer-act loop. It observes the current screen, checks whether the pending action is valid under the live state, maps the recorded target to a current GUI element, and dispatches the action only when the match is sufficiently confident. For target mapping, Flow uses anchor-based transfer. It compares the recorded screen and the live screen through element-level visual, structural, geometric, and interaction features, then uses stable neighboring elements as anchors to locate the current target. The final target is selected using local layout relations, tree structure, element compatibility, and confidence-margin checks. When the evidence is weak, ambiguous, or blocked by runtime conditions such as dialogs or advertisements, Flow abstains and hands control back to recovery logic or VLM-based action generation.

Together, these mechanisms make adaptive replay possible under dynamic mobile execution. The Omni representation layer increases the chance that useful parts of a prior execution can still be reused after workflow divergence. The Flow execution layer checks and transfers each recalled action against the live screen before it is performed. This separation lets OmniFlow exploit repeated structure in mobile tasks while preserving the underlying GUI agent as the fallback path for novel, unsafe, or semantically ambiguous states.

We implement OmniFlow as a plug-in cache and execution layer for Android GUI agents. Our current evaluation shows that OmniFlow can convert successful prior executions into zero-model-call Function replay on matched tasks, while retaining fallback for uncertain states. On the accepted AndroidWorld replay set, OmniFlow completes all 30 validator-scored reuse runs with zero online model calls and zero online model tokens. For cross-device UDEG retrieval, OmniFlow achieves 98.63% Hit@1 and 100.0% Hit@3 on the currently reviewed subset, with sub-millisecond lookup latency. These results suggest that adaptive replay is a practical complement to VLM-based GUI agents: step-wise reasoning handles novel or ambiguous states, while OmniFlow serves recurring GUI fragments through verified reusable Functions.

This paper makes the following contributions.

- We identify repeated mobile GUI execution as an important serving workload for LLM/VLM-powered GUI agents, and characterize the workflow-level and GUI-state-level dynamics that make existing acceleration, memory, and replay methods insufficient.
- We design OmniFlow, an adaptive replay system that decomposes successful executions into reusable state-indexed Functions and executes them through checked live-screen grounding.
- We implement and evaluate OmniFlow on repeated mobile GUI tasks, showing that reusable Functions can reduce online model invocation cost while preserving fallback to the underlying GUI agent.
- OmniFlow's replay path requires no VLM and no embedding model: state matching and action transfer rely entirely on feature-engineered page vectors derived from accessibility-tree structure, so the full reuse pipeline runs on mobile CPU hardware with zero online model tokens on a cache hit, retaining only a lightweight chat LLM for cache-miss fallback.

2 Background and Motivation

In this section, we review recent advances in mobile GUI agents powered by emerging LLMs and VLMs, along with the high execution costs associated with the traditional execution pipeline. We then examine efforts to reduce the runtime costs of such agents and discuss their limitations, which motivate the design of OmniFlow.

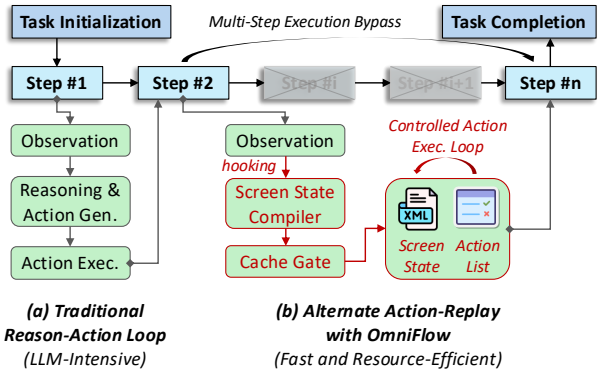


Figure 1. Overview of OmniFlow workflow. The framework dynamically alternates between a traditional LLM-intensive reason-action loop (a) and a fast action-replay mechanism (b) to accelerate task execution. By hooking observations and querying the memory, the system recognizes familiar screen states and adaptively replays a cached list of atomic actions.

2.1 High Execution Costs of GUI Agents

Mobile GUI agents typically operate in an observe–reason–act loop [19, 22, 24, 29]. As shown in Figure 1, at each step, the agent perceives the current screen, invokes an LLM or VLM to determine the next GUI action, and executes the action in the environment. Although this workflow enables flexible reasoning over open-ended tasks, it introduces substantial runtime overhead because model inference is repeatedly triggered during long interaction sequences. The cost accumulates with each additional step, leading to increased latency and computation during task execution [1, 5, 28, 30, 31]. Prior work reports that existing mobile agents often require more than 10 seconds per step, 1–2 minutes, and about $1 monetary cost per task on average [4, 5]. More broadly, planning and reflection alone can account for 75%–94% of end-to-end latency in GUI agents [1]. These costs become more pronounced when the step-wise model inference runs directly on mobile devices with strict memory, power, and thermal constraints [4, 5, 27].

However, real-world mobile usage rarely consists of purely open-ended tasks. In practice, users repeatedly perform similar routines, such as checking routine information, retrieving specific content, completing fixed navigation paths, and filling structured form-like workflows. Prior studies characterize smartphone interaction as highly habitual, with repetitive checking behavior and app-launch patterns that are strongly shaped by recurring contexts such as time and location [17, 26]. Public mobile GUI datasets and benchmarks also reflect a similar pattern. For example, the largest scale mobile GUI dataset to date, AITW, contains 715K episodes but only around 30K unique instructions [20]. Therefore, although per-step reasoning for open-ended tasks is crucial for high task completion rates, many realistic mobile-agent workloads are better viewed as repeated executions of recurring workflow patterns.

Table 1. Execution acceleration methods for mobile GUI agents. The columns capture whether a method (1) requires prior exploration on GUI states, (2) utilizes a cached memory without invoking LLM/VLM again, and (3) transfers its memory to dynamic and heterogeneous execution environments.

Method	W/ App Exploration	W/ Model Invocation	Cross-Env Transferability
RPA/script replay [8, 12, 15, 16]	✓	✗	✗
Memory-based [11, 24, 29]	✓	✓	✗
Program-guided [21, 23, 25, 31]	✓	✗	✗
Per-step acceleration [2, 5, 13, 18]	✗	✓	✗
OmniFlow	✗	✗	✓

[This paragraph is the limitations of existing replay mechanisms; it also introduces massive new concepts such as coordinate replay and selector replay, which have not been introduced ahead.] This reuse problem is constrained by the observation interface itself. Mobile GUI agents usually observe a screenshot, an Android accessibility tree, or both. The accessibility tree reliably exposes element bounds and hierarchy when elements are observable, while semantic attributes such as `resource-id`, `text`, and `content-desc` only appear when developers provide them. As a result, repeated execution cannot assume that every target has a stable selector: coordinate replay breaks under device-size and layout changes, and selector replay fails on missing annotations, custom-rendered widgets, WebViews, obfuscated resource identifiers, and unlabeled container controls. The same instability limits graph-level reuse. A conventional UI transition graph provides a useful state structure, but its nodes are often tied to raw XML or unstable attributes, and its edges record dispatched actions and state transitions rather than verified task effects. The missing object is a callable transition: a UTG edge can record that one action once moved an app from one observed state to another, but it does not by itself package runtime arguments, source-context transfer, or terminal/effect verification as an API-like operation. Such a graph is therefore difficult to reuse as a cross-device and cross-user execution cache under dynamic layouts and user-specific content.

2.2 Reducing Runtime Costs

Prior studies on reducing the runtime costs of GUI agents primarily fall into several categories, which are summarized in Table 1.

Robotic process automation (RPA)-based automation systems treat mobile tasks as fixed workflows by recording and replaying user demonstrations [8, 12, 15, 16]. These systems construct replayable execution traces through human-driven app exploration before task execution. However, the resulting memory is tightly bound to the recorded execution

path, GUI layout, and entry state. As a result, even minor changes in the live GUI state, such as screen modifications or layout updates, can cause the recorded execution to fail.

Memory-based GUI agents reduce runtime costs by reusing memory constructed from prior app exploration or first-time task execution. For example, M3A [19] stores prior actions and their reasoning in prompts to guide later execution. AutoDroid [24] and AppAgent [29] construct app-specific memory, such as UI transition graphs, through automated exploration and retrieve this memory during execution. MobileGPT [11] extends this idea with hierarchical app memory that stores reusable procedures and subtasks summarized from prior executions. MobiMem [14] reduces repeated GUI-agent reasoning by reusing prior executions through experience memory, which abstracts task traces into reusable templates, and action memory, which caches low-level action sequences for direct replay. **Program-guided agents** [21, 23, 25, 31] further reduce runtime costs by translating task knowledge or prior demonstrations into executable scripts. These scripts allow repeated GUI operations to be completed through deterministic program execution with fewer model invocations. Together, memory-based and program-guided agents replace parts of VLM-based step-wise action generation with previously validated memory or executable operations. However, these methods still rely on memory constructed from prior app exploration, user interaction, or first-time execution, which can introduce privacy concerns and limit transfer across users, devices, app versions, and unseen environments.

Another line of studies, **per-step inference acceleration methods**, retain the traditional observe–reason–act loop while reducing the cost of each individual decision. V-Droid [5] replaces auto-regressive action generation with verifier-based ranking over a discretized action space, which substantially reduces latency. ShowUI [13] accelerates VLM inference through UI-guided visual token selection. Other studies accelerate execution by pruning redundant visual or historical context before model inference, such as masking-based GUI element pruning, history compression [2], and position-preserving UI token selection [18].

To summarize, existing approaches still face several limitations. RPA-based systems and script replay provide executable workflows, but they are fragile when the live GUI state diverges from the recorded execution. Memory-based and program-guided agents reduce repeated reasoning, but they often rely on substantial app exploration and provide limited support for workflow-level adaptation. Per-step inference acceleration methods reduce the cost of each individual decision, but they still require repeated VLM-based step-wise action generation in long interaction loops. These limitations motivate us to build OmniFlow for generalizable memory reuse across dynamic app environments.

2.3 Goals and Challenges

Our goal is to design a memory system for mobile GUI agents that can reuse past executions in dynamic app environments while allowing flexible switching between memory-guided execution and VLM-based step-wise action generation. This requires memory to support both workflow-level adaptation, where the execution path may change, and state-level adaptation, where a memorized GUI state may appear differently from the live GUI state. A key design principle is to keep the replay surface minimal, because prior systems often introduce separate runtime mechanisms for each variation case, such as coordinate transfer, screenshot/XML embedding, and selector-specific recovery, which are costly and brittle on mobile devices; OmniFlow exposes compact cache capabilities to the agent and strengthens state lookup, action transfer, and verification with mechanically verifiable evidence derived from XML structure. The core observation is that mobile apps expose a fixed-action subspace: within a ReAct loop, recurring tasks often contain high-frequency repeated Functions, such as opening a search page, entering a query, filling a form, or confirming a save. We call such reusable units `Functions`. This observation suggests storing them as persistent Functions that can be updated over time, so the agent can skip reasoning and execute directly when the same situation recurs. The key challenges are how to preserve these Functions as reusable Functions and how to execute them reliably across devices.

Reuse Adaptation. The first challenge is representing past executions at a granularity suitable for flexible reuse. Existing memory systems for GUI agents often store a complete long-horizon trajectory or split it into fixed subtasks, then guide the agent to follow the recorded execution path. This design is fragile in real mobile apps, where workflows frequently change due to expired login sessions, pop-ups, loading delays, or permission dialogs. A coarse-grained memory may fail even when most parts of the historical execution remain reusable. This calls for a finer-grained memory representation that can reuse matched Functions while allowing the agent to fall back to VLM-based step-wise action generation when the live workflow diverges. We address this challenge by organizing reusable state around UDEG nodes and task-carrying operations as Functions. A UDEG node captures a reusable GUI state, while a Function captures a callable operation that can be executed when the live state is compatible. This structure enables partial reuse, local recovery, and flexible coordination between memory-guided execution and VLM-based step-wise action generation.

Adaptive Action Grounding. The second challenge is applying a memorized operation to a live GUI state safely. Even when the live GUI state corresponds to a memorized GUI state, the target GUI component may shift due to screen size, font scale, list ordering, or dynamic content. Directly replaying coordinates or fixed selectors can trigger the wrong

Table 2. Single-observation state-description cost on mobile hardware. Repeated model inference at query time reintroduces the overhead that a reuse system is designed to eliminate. OmniFlow's PageVector lookup uses zero model tokens and completes in under 5 ms.

Location	Backend	Input	Tokens	Time
On-device	Llama.cpp + 0.8B-W4 + CPU	screenshot	~2,500	60 s+
On-device	MNN-LMM + 0.8B-W4 + CPU	screenshot	~2,500	10 s+
On-device	Omniinfer + 0.8B-W4 + NPU	XML	~1k–10k+	3 s
Cloud	Qwen-VL-Plus	screenshot	~3,200	3 s
OmniFlow	**PageVector**	**XML features**	**0**	**4.13 ms**

operation, which is especially risky for mobile agents because mistakes may modify or delete user data. Reuse therefore requires state-level adaptation. The system must first determine whether the live GUI state matches a reusable memory node, then map the memorized action target to the correct executable target in the live GUI state. This requires grounding memory in semantic GUI structure instead of relying only on recorded visual positions or raw widget attributes.

Minimal model dependency on device.

The third challenge is that device-level constraints limit the feasibility of repeated model inference.

3 System Design

3.1 Architecture and Workflow

Figure 3 gives the end-to-end workflow. OmniFlow treats repeated mobile GUI execution as API synthesis over black-box apps. Mobile apps expose official APIs to developers, but GUI agents usually interact with opaque screens and accessibility trees. OmniFlow therefore synthesizes API-like `Functions` from successful GUI executions and stores them in a Function Store. Each Function packages public arguments, start-state conditions, source-context evidence, an executable action body, and terminal/effect checks. The Function Store naturally induces a sparse User-Driven Execution Graph (UDEG): action-compatible GUI-state families become nodes, and callable Functions become executable edges. The graph is an indexing and evidence structure for Function invocation rather than a full-app transition model or a global planner. OmniFlow consists of two layers. **(1) Omni representation layer** manages the Function Store by converting successful runs into callable Functions and UDEG state evidence, and exposes a recall interface while keeping raw histories out of online reasoning. **(2) Flow execution layer** consumes a recalled Function and grounds its steps in the live GUI through a closed-loop observe–check–transfer–act stack.

The two layers communicate via a unified MCP protocol: Omni exposes a query interface, and Flow retrieves Function candidates plus compact context before executing them. Both layers act as wrappers around the existing agent-device loop. Omni hooks before the host planner (recall), and Flow hooks before the physical `act()` interface (checked execution), enabling OmniFlow to integrate as a modular cache and execution layer without replacing the host agent.

3.2 Omni Representation Layer

Omni converts successful GUI executions into reusable Function assets. A conventional UTG captures useful state/action structure, but a replay system needs more than a historical edge. It must localize the current state, aggregate semantically equivalent page variants, bind runtime arguments, transfer the recorded action from source context to target context, and verify the intended effect. OmniFlow upgrades the UTG structure by making the edge a callable `Function`; the UDEG then supplies the state index and evidence needed to invoke that Function safely.

The representation layer has two objects. **(1) UDEG nodes** are action-compatible page clusters that provide localization and semantic aggregation over GUI-state variants. **(2) Functions** are API-like replay primitives extracted from successful trajectories, and they are the core assets in the Function Store.

Node. Raw GUI inputs, including pixel-level screenshots, textual view hierarchies, and raw text, can vary with screen size, density, app version, layout, language, and runtime state. They also increase online reasoning cost and may expose private user information. Therefore, a UDEG node stores stable visual, structural, geometric, and interaction features, while raw GUI content is discarded after feature projection.

Figure **??**(a) shows how raw GUI observations are converted into UDEG nodes. Omni first decomposes the textual view hierarchy into UI elements. For each element, it builds a 64-dimensional feature vector from two sources: screenshot-derived visual cues, such as icons, labels, and clickable regions, and view-hierarchy-derived cues, such as widget class, interaction flags, bounds, and tree position. Raw text is projected into privacy-preserving features and then discarded. The element vectors are pooled into a compact page vector for approximate lookup. For each UI element e_i, Omni builds an element vector

$$\mathbf{v}_i = \phi(e_i),$$

where $\phi(\cdot)$ combines visible cues and accessibility-structure cues: content hash, element type, interaction flags, subtree shape, hierarchy, resource-id hints, and normalized geometry. Omni then groups elements into UI subspaces, each defined by a predicate over the element and its page context:

$$\mathcal{S}_k(X) = \{e_i \in X \mid c_k(e_i, X) = 1\}.$$

For each subspace, Omni computes a weighted vector:

$$\mathbf{p}_k(X) = \frac{\sum_{e_i \in \mathcal{S}_k(X)} \alpha_{ik} \mathbf{v}_i}{\sum_{e_i \in \mathcal{S}_k(X)} \alpha_{ik}}.$$

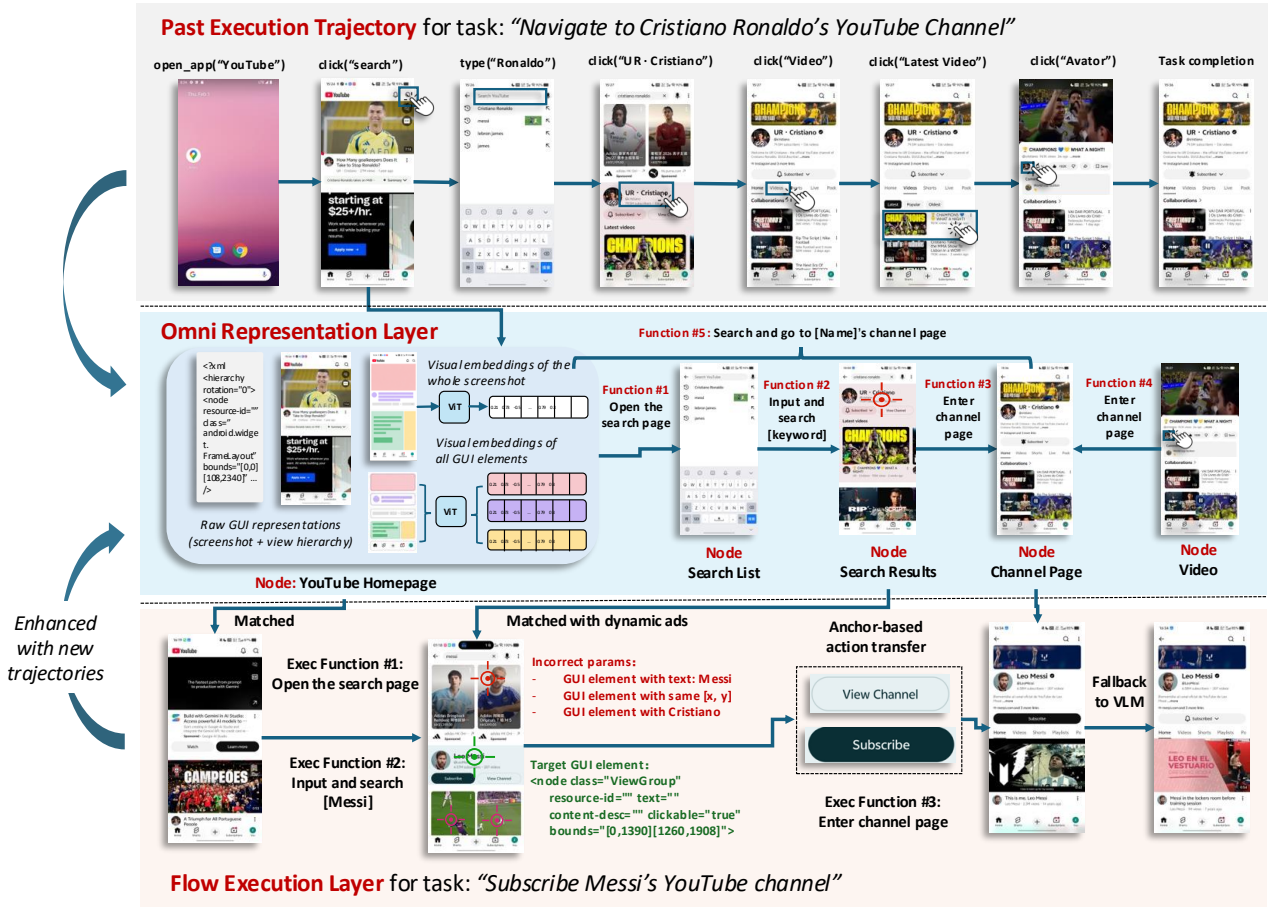


Figure 2. Demonstration of how a task execution trajectory is transformed into memory and how that memory is adaptively utilized in online execution environments.

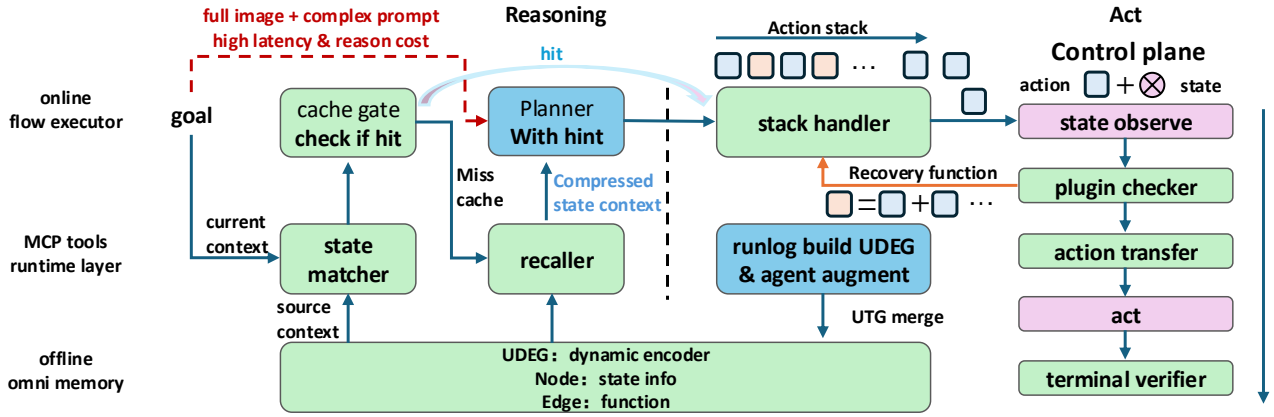


Figure 3. System design overview. OmniFlow builds a Function Store whose assets are API-like Functions synthesized from successful GUI executions. The store induces a sparse UDEG: privacy-safe GUI-state nodes index where Functions can be invoked, and Function edges record executable task effects. Omni recalls a compatible Function through this state index; Flow executes it through a single observe-check-transfer-act stack and writes accepted run logs back into the Function Store.

The page vector is the weighted concatenation of these subspace vectors:

$$\mathbf{p}(X) = \frac{[w_1\mathbf{p}_1; \ldots; w_K\mathbf{p}_K]}{W_P}, \qquad W_P = \sum_{k=1}^{K} w_k.$$

The element-level weights α_{ik} decide which elements matter inside a subspace, while the page-level weights w_k decide which subspaces matter for page matching and action transfer.

Our implementation uses fixed feature-engineered weights and eight fixed subspaces; the same formulation naturally supports learned weighting functions. Appendix A gives the corresponding pseudocode. The page vector serves as the state signature, and the retained element-level features provide the anchors needed by Flow to transfer recorded actions to the current screen. This representation gives Omni four properties needed for reusable execution. First, it forms a privacy-safe shared payload because raw screenshots, raw XML, and raw text are not stored. Second, it preserves relation-aware action evidence: element vectors retain local structure, geometry, and interaction cues needed to bind a recorded target to the current screen. Third, it makes online matching cheap, reducing page lookup and element transfer to vector search and pairwise matrix scoring. Fourth, it covers common failure cases of coordinate and selector replay, including layout shift, missing or duplicated attributes, and targets whose executable parent differs from the labeled child node. Node matching is based on action compatibility rather than visual similarity. Two observations can share the same UDEG node when their UI elements support the same key actions and those actions can be safely transferred across layout, text, device, or density variations.

Function. To reuse prior executions without replaying fragile full trajectories, Omni represents each reusable task fragment as an API-like `Function`. A single accepted run log is not stored as one monolithic replay trace. Omni decomposes it into one or more reusable `Functions`, and Flow later executes only the Functions whose start state and arguments match the live task. This lets one recorded execution support multiple later partial reuses instead of requiring the live task to follow the original end-to-end trace. A Function keeps only the task-relevant part of a successful trajectory and stores it as a reusable executable unit. It stores a start condition, an argument schema, and an action sequence, together with compact evidence from the source execution. The start condition specifies when the Function can be applied; the argument schema abstracts concrete values such as user input, selected items, or target names into reusable parameters; and the action sequence stores the operations needed to execute the Function.

Functions can also be composed across reuse boundaries. A Function body may contain atomic device actions, external tools, scripts, and calls to other Functions. Nested Function calls allow previously accepted Functions to be reused inside larger tasks. The planner or cache gate can therefore invoke a Function through a callable interface: select the `Function`, bind its arguments, and let Flow execute the resulting action stack under state checks and action transfer.

Recall. Recall uses the UDEG as the state-indexed view of the Function Store: the current GUI observation is encoded as a page vector, matched to an action-compatible UDEG node, and used to retrieve candidate Functions attached to that node. Recall then uses a cache gate to decide whether a matched Function can bypass planner reasoning and be executed by Flow. The gate is necessary because GUI actions may change user-visible state. A Function is returned for direct execution only when three checks pass: the live GUI observation matches the stored start state, the Function preconditions pass Flow's state checks, and the Function arguments can be resolved uniquely. If these checks pass, Omni returns an executable Function package. If any check fails, Omni returns a bounded planner hint, injects the Function into the planner's action set as a dynamic callable, attaches compact page-history analysis, and leaves the next decision to the planner.

A bounded planner hint contains only the matched node summary, candidate Function names, page-history analysis, missing preconditions, and unresolved arguments or GUI targets. It excludes raw screenshots, full traces, and long action histories. This design reduces the VLM reasoning overhead during planning because the planner receives structured reuse candidates instead of having to reinterpret past trajectories, screenshots, or textual summaries. We therefore evaluate recall by the returned context size and the lookup latency before planner invocation.

Update. Write updates the Function Store and its induced UDEG from execution logs. After the run finishes, Omni analyzes the accepted parts of the logs offline, without incurring additional overhead to the online process.

For a successful run log, Omni performs four offline updates. It analyzes the run semantics and decomposes the trajectory into Functions; converts observed GUI states into page vectors; merges them into the existing UDEG library; and, for new nodes or Functions, applies agent-based semantic augmentation and analysis before updating execution evidence. Failed or rejected Functions also reveal missing preconditions, unstable transitions, unresolved targets, or runtime deviations. Omni uses this evidence to refine cache-gate checks and update recovery rules in Flow's checker.

Together, these designs make Omni a compact representation layer that converts historical executions into API-like Functions and the UDEG state evidence needed to recall them. Flow then consumes these artifacts for verified execution, action transfer, and runtime adaptation, which will be described in the next section.

3.3 Flow Execution Layer

Flow is the execution layer that grounds recalled Functions and VLM-generated actions on the live device. It receives two kinds of inputs: an executable Function package returned by Omni, or a single action generated by the host VLM agent. For both inputs, Flow runs the same closed loop: observe the current GUI state, check the pending action, transfer the recorded or generated target to the live screen, dispatch one concrete `act`, and observe again before advancing. When Omni's recall gate returns a Function package, Flow executes the matched Function and skips VLM action generation for

that Function; when recall returns only bounded context, the host agent uses that context with the current state to generate the next action.

Plugin checkers. During execution, Flow invokes lightweight checker plugins between observation and action dispatch. Each checker defines a configurable trigger over the current GUI state, the pending action, or the current execution context. Checkers can be global, node-specific, or Function-specific. Common checkers detect whether a dialog covers the target region, whether an advertisement blocks interaction, whether the page is still loading, whether the expected text field is editable, or whether a destructive action requires confirmation. Local recovery is handled by these checker decisions rather than as a normal phase of every Function. Checkers also provide a lightweight write-back path for historical failure experience: failed or rejected run logs can add new triggers and small recovery policies without changing the main Function body. A checker can attach a fixed action set to a page, such as dismissing a known dialog or waiting through a loading state, making it a convenient extension point for page-local control. When a trigger fires, the checker returns one control decision for the current execution context. The decision can be continue, wait, run a short recovery action, or fall back to VLM action generation.

Action transfer. Action transfer maps a recorded source action in a recalled `Function` to an executable action on the live GUI state:

$$\texttt{transfer} : (S, e, T) \mapsto e',$$

where S is the recorded source page, e is the source action target, T is the current page, and e' is the target element to execute on the live device. The design follows a common property of mobile GUI tasks. Repeated actions are usually defined by UI structure and local context, while absolute coordinates alone are often unstable. A button for adding a contact or submitting a form may move across devices, app versions, or languages, but its role, neighboring labels, input fields, and local layout relations often remain informative.

Flow uses anchor-based transfer to rebind recorded targets to the live screen. Coordinate replay and selector replay rely on a single unstable cue: the recorded point or the target element's own attributes. This is fragile on mobile GUIs, where elements often lack stable `resource-id`, visible text, or accessibility names, and where layouts shift across executions. Flow instead treats transfer as relation-aware matching between the recorded and current UI trees. It scores source and current elements with UDEG-node element vectors, selects high-confidence one-to-one matches as anchors, and lets each anchor vote for a compatible current target by projecting the recorded action through the action–anchor relation. Each vote combines action similarity, anchor similarity, and action–anchor distance consistency from bounds and parent links. This makes transfer depend on stable local and global UI structure, such as cards, search bars, tab bars, section headers, and navigation controls, while avoiding an early commitment to global anchors that do not explain the current action.

Algorithm 1 Anchor-based action transfer for one recorded action

Require: source page S; current page T; recorded source action element e
Ensure: executable target action or Abstain

$M \leftarrow$ pairwise element-similarity matrix between S and T
$A \leftarrow$ high-similarity one-to-one pairs (a, a') from M
$V \leftarrow$ empty vote table over elements in T
for all $(a, a') \in A$ **do**
  $\hat{x} \leftarrow$ project e through the path $e \rightarrow a \rightarrow a'$
  $C \leftarrow$ compatible current elements near $\hat{x}$
  for all $e' \in C$ **do**
    $r \leftarrow$ relation consistency between (e, a) and (e', a')
    $v \leftarrow \textsc{Sim}(e, e') \cdot \textsc{Sim}(a, a') \cdot r$
    $V[e'] \leftarrow V[e'] + v$
  end for
end for
$c^* \leftarrow$ highest-vote element in V
if c^* has enough vote support and margin **then**
  return recorded action rebound to target element c^*
else
  return Abstain
end if

Algorithm 1 shows the transfer procedure. Rather than matching globally, Flow starts from the action's local context and expands outward. For each candidate anchor pair (a, a^*), the vote cast for a current element e' is weighted by three terms: $\textsc{Sim}(e, e') \cdot \textsc{Sim}(a, a^*) \cdot \textsc{Dis}(e, a)$, where Sim is element-vector similarity and $\textsc{Dis}(e, a)$ measures how consistently the geometric and tree-structural relation between action and anchor is preserved across screens. An anchor with high global similarity but poor local consistency is therefore down-weighted. Flow selects the top-voted element only when it has sufficient support and a clear margin; otherwise it abstains and emits a recovery signal.

4 Implementation

wrap layer. OmniFlow is implemented in both Kotlin and Python so the cache layer can be deployed at different integration points without changing the host agent. The Kotlin implementation runs close to Android: it binds to the accessibility service and input APIs, collects observations, dispatches physical actions, and can host the local UDEG index on device. The Python implementation targets host-side agents, evaluation harnesses, and shared stores: it exposes the same Function recall, Function execution, and run-log ingestion tools while hosting the shared UDEG index and Function store. Both implementations use the same `run_log` schema, `Function` format, and MCP tool definitions, so a run

log collected from a human, a VLM agent, or a benchmark harness enters the same write-back path.

A central integration goal is zero modification to the host agent. OmniFlow attaches at the physical GUI I/O boundary—`observe`, which reads the current screen state, and `act`, which dispatches one device action. Everything above this boundary remains the host system's planner and task loop. The three MCP tools wrap this boundary into a cache layer: *Function recall* intercepts before planning, *Function execution* drives the device after a Function is selected, and *run-log ingestion* writes accepted executions back after completion. The same tool boundary is the measurement surface used in §5.

hybrid retrieval and cache gate. The UDEG node index holds one normalized page vector per unique GUI state. The Python server tier uses FAISS for high-throughput retrieval over shared cross-user pools; the Kotlin on-device tier uses Annoy's memory-mapped tree index for low-footprint read-heavy access. Both backends support the same cosine similarity interface and can operate fully locally, with no cloud round-trip required. Function recall uses a two-stage strategy. A dense stage queries the index to identify the candidate UDEG node family by page-vector similarity. A sparse stage ranks candidate Functions by matching the current task instruction against each Function's *description* and *argument schema*: a Function scores high when its description keywords overlap with the instruction and its arguments can be resolved from the instruction without ambiguity. These two fields are sufficient to disambiguate Functions that share the same UDEG node—for example, `archive_email({sender})` and `send_email({recipient, content})` attached to the same inbox node are separated by both description and argument structure. The cache gate returns a Function for direct execution when the node similarity and Function score both clear their thresholds; candidates below threshold are returned as planner hints instead.

Offline Function extraction. Omni converts an accepted `run_log` into a `Function` in two offline steps. The first step is fully local: Omni segments the trajectory at app-boundary and terminal-action cuts, builds UDEG nodes from the XML observations at each segment boundary, and encodes element vectors for the action targets. The second step applies a single offline LLM call per segment. The LLM receives the task instruction and the action sequence (action types and concrete values, without raw screenshots or XML) and performs three operations: it identifies which concrete values are task-specific arguments to be rebound at reuse time (e.g., lifting a typed phone number into `{phone_number}`), it assigns a name and short description to the Function, and it confirms or adjusts the segment boundary. The result is a typed argument schema stored with the Function; all subsequent reuse is argument-fill only. At reuse time, a second lightweight call resolves the live argument bindings from the incoming task instruction.

Flow execution loop. Before a concrete device action is issued, Flow runs a lightweight pre-action path: fresh observe, Function recall, cache-gate checking, and either cached execution or planner handoff. The fresh observation is collected by the device adapter as an accessibility traversal plus screenshot. Recall queries the local UDEG index, ranks candidate Functions, and passes them through the cache gate. On a hit, Flow bypasses the host planner, expands the Function into a serial action stack, and executes each pending action through observe-check-transfer-act. On a miss, Flow injects the candidate Functions and compact page-history context into the host model; each Function is exposed as a dynamic tool alongside the normal atomic action tools. This pre-action path is designed to stay within roughly 100 ms for typical reuse queries, so the runtime bottleneck is the actual GUI action and page transition rather than preprocessing.

5 Evaluation

Our evaluation is divided into two parts: a comparative study and an ablation study. The comparative study measures whether a human-collected source execution can be reused on similar tasks with lower latency and online model cost. The ablation study isolates the contribution of action transfer, UDEG localization, and representation overhead.

5.1 Experimental Setup

Tasks Setting. We use two complementary workloads. First, we use AndroidWorld as a validator-backed **system-app workload**. This set contains 30 accepted replay task instances covering system-level apps and settings, such as SMS/Phone, Contacts, Camera, Recorder, Clock, Bluetooth, Wi-Fi, and brightness controls. These tasks are important because system apps vary across Android versions, OEM skins, device initialization states, and permission flows. Each source execution is successfully recorded, and each reuse execution is scored by the AndroidWorld validator with a different replay seed from the source run.

Second, we collect an **app-level reuse workload** from common third-party apps. This workload focuses on repeated workflows such as search, filtering, opening a target page, filling forms, switching tabs, dismissing popups, and interacting with dynamic list/card interfaces. These cases stress WebView pages, repeated content, account-specific state, and layout shifts that are less central in system-app benchmarks.

We run the evaluation on a mix of real devices and virtual machines, including a Oneplus 12, a Pixel 9 Pro, a Vivo X300 Pro, and a Magic Fold V5. These devices cover different Android versions, screen sizes, and hardware profiles, allowing us to evaluate cross-device reuse rather than same-device replay only.

Device	OS Family	Display	Resolution	Form Factor
OnePlus 12	OxygenOS	6.82"	3168×1440	Phone
Pixel 9 Pro	Android	6.3"	2856×1280	Phone
vivo X300 Pro	OriginOS	6.78"	2800×1260	Phone
HONOR Magic V5	MagicOS	7.95"	2352×2172	Foldable

We use qwen-vl-plus as agent backend for all systems, including the baselines, to ensure a fair comparison of the reuse strategies rather than differences in underlying model capabilities.

Baselines. We compare OmniFlow against three categories of baselines.

We use the same source run log as the memory source for all baselines. Each run log is a sequence of timestamped device observations and actions, recorded during a successful execution of the target task on a source device to simulate a human operator's first attempt at the task. All methods can offline augment and analyze the run log, because only online cost is counted in replay experience.

Native Replay. The native replay baseline directly replays the source run's action sequence on the target device, with easy reuse or adaptation. They are expected to have low latency and token cost but may struggle with cross-device differences. We choose two representative native replay methods: (1) **coordinate scaling**: scale the source action's screen coordinates to the current screen and tap at the scaled location; (2) **selector lookup**: locate the nearest selector match in the current XML tree, such as resource-id or text.

Step-wise agent. M3A is a original step-wise agent in androidworld baseline. It receives the current device observation at each step and reasons about the next action to take.

Memory-based agents. For AutoDroid, we convert the source run log directly into its native memory format. For MobileGPT, we evaluate two variants: (1) **cold start + warm start**: MobileGPT first performs autonomous cold-start exploration to populate its memory, then executes the target task in warm-start mode; (2) **warm start from source run log**: we convert the source run log directly into MobileGPT's memory format, bypassing the exploration phase. This conversion mirrors natural human operation: a user who has completed a task once has already observed every intermediate screen and action, so the per-step screenshots and view hierarchies captured in the source run log provide exactly the rich observation data these agents require. All reuse runs receive no human assistance and are executed from a fresh device state.

Evaluation protocol. The source run is used only to construct reusable state; it is not counted as target-run success. A target run starts from a fresh device state and asks the system to serve the same task family without additional human demonstration. A run is counted successful only when the terminal task effect is correct (e.g., contact created, message sent, alarm set); intermediate step order is not scored. For AndroidWorld, success is determined by the official validator.

Table 3. AndroidWorld 30 replay snapshot. Nits: (1) How we select the original seeds, and how the target seeds are selected; (it is better to scale to more than 3 target seeds to get robust performance). (2) MobileGPT: Separate to cold and warm setups (do not accumulate total time, calls, tokens). (3) Can we add another setup for MobileGPT: add ground-truth memory for it?. (4) Two setups for OmniFlow: agent without memory and agent with memory. (5) Other approaches to filter GUI pairs for micro-benchmark in sec 5.3. (6) Start end-to-end eval of MobileGPT tasks on different devices. (7) Enhance this section with on-device performance eval.

System	SR	Total time	Calls	Tokens
M3A (Qwen-VL)	70.0%	146.4 min	415	2,120,446
AutoDroid	36.7%	64.9 min	81	62,029
MobileGPT (cold+warm)†	53.3%†	85.2 min	576	476,257
OmniFlow	**100.0%**	**6.1 min**	14	5,275

MobileGPT cold+warm time = 63.4 min source exploration + 21.8 min target run.

For all other tasks, success is determined based on the final device state and task outcome.

Metrics. The primary metrics are task-level success rate (SR), average fallback count per run (the number of steps that could not be served from cache and fell back to the host agent), total token consumption, and end-to-end latency.

5.2 Comparative Study: End-to-End Replay Performance

Table 3 reports the current AndroidWorld 30 comparison results. OmniFlow is evaluated on the accepted Function replay set with changed collect/replay seeds. M3A, AutoDroid, and MobileGPT are online baselines that reason during the target run; the AutoDroid and MobileGPT rows use the same OOB `/get_state` state-capture boundary while keeping their native policies and action execution.

Our OmniFlow's cache gate successfully recalls the correct reusable state and function for all tasks. while parameters slotting is necessary and not important component for GUI tasks. On execution we do not use any other model calls. MobileGPT needs cold start to confirm the reusable state, and with a raw run log as memory, it can not perform well.

Moreable we finish our OmniFlow-bench on our four devices, and the results show that OmniFlow can achieve high cross-device reuse rates with low latency and token cost, while baselines struggle to adapt to cross-device differences.

5.3 on device evaluation

5.4 Ablation Study

Experimental design. GUI grounding is a key challenge for cross-device reuse. We construct a dedicated ablation study to evaluate the effectiveness of action transfer. We collect a set of action pairs from the Rico dataset and original run logs, where each pair represents similar tasks executed

Table 4. Registered action-transfer comparison. Final rates require the expanded action-pair gold set.

Strategy	SR	Avg Time
OmniFlow transfer	gold pending	
Selector lookup	gold pending	
Coordinate scaling	gold pending	

Table 5. Cross-device representation recall and encoding cost.

Method	Hit@1	Hit@3	Avg encode time	Avg tokens
VL-embedding	69.44%	87.50%	188.69 ms	1,009
PageVector	98.63%	100.0%	4.13 ms	0

on different devices, but direct replay fails due to differences in screen layout and element positions. We design three strategies to attempt to transfer these actions: (1) **action transfer**, which uses our proposed method to remap actions based on UDEG localization and anchor relocation; (2) **selector lookup**, which attempts to find the nearest matching selector (resource-id or text) in the current XML tree; and (3) **coordinate scaling**, which scales the source action's screen coordinates to the current screen and taps at the scaled location.

5.5 Representation and Overhead

We evaluate the representation layer independently from final task execution. The goal of this experiment is to test whether UDEG can group cross-device observations of the same action-compatible GUI state and retrieve the correct cluster for a held-out page.

We compare two representation methods: (1) a screen-only VL embedding baseline that encodes the raw screenshot using a vision-language model, and (2) our designed UDEG/PageVector method that combines visual and structural information. The results show that our method significantly outperforms the baseline, achieving a much higher cross-device representation recall. This is because the baseline only captures the visual features of the screen, while our method captures both visual and structural information, allowing it to more accurately identify similar page states across different devices and achieve higher cross-device reuse rates. In 10k node UDEG, one-time localization recall use only

6 Discussion

Execution flywheel: every run refines the shared cache. A key property of OmniFlow is that the shared Function cache improves with natural usage—no dedicated exploration phase is required. Each accepted run is written back through the same write-back path: Omni projects the new observation into the existing UDEG node family, adds the run as additional evidence, and refines the Function's precondition and action evidence offline. The consequence is that the same UDEG node accumulates observations from diverse devices, users, app versions, and screen configurations over time. A Function that was originally recorded on one device and screen size gradually acquires anchor evidence from foldable displays, different OEM skins, and varying font scales. This makes subsequent cache hits more robust—not because the system was trained on more data, but because the PageVector and anchor evidence naturally span a wider layout manifold. The cost structure follows: the per-Function online model cost is paid once, on the first execution that discovers the Function. Every subsequent cache hit amortizes that cost across all future users. In workloads where the same recurring Function is triggered by many users, the marginal online model cost per invocation approaches zero as the cache matures. This flywheel is absent in existing memory-based systems that build per-user, per-app memory from dedicated exploration: each new user resets the cold-start cost, and the memory does not cross-fertilize across users or devices.

Our efficient page matching and anchor relocation algorithms make repeated write-back practical at runtime. As a result, every successful execution can be incorporated into the shared cache with little additional overhead, allowing execution knowledge to accumulate continuously through normal usage.

7 Related Work

Mobile UI record-and-replay. Record-and-replay systems are the closest prior line to OmniFlow's core problem. Systems such as SUGILITE [12] and Reran [8], as well as mobile testing and automation tools such as Maestro [15] and uiautomator2 [16], turn user or tester interactions into executable traces. These traces are cheap to run because they do not require per-step model inference, but they are usually tied to a recorded entry state, a fixed screen geometry, or stable UI selectors. Small changes in layout, list content, device size, app version, or runtime dialogs can therefore make the trace click the wrong component or abort. OmniFlow keeps the record-and-replay execution benefit, but changes the replay unit: a successful run log is converted into a Function whose actions carry source GUI context. During replay, Flow observes the live screen, checks blocking conditions, transfers the recorded action target to the current GUI, and dispatches the action only when the transferred target is sufficiently supported.

Step-wise and verifier GUI agents. VLM-based mobile GUI agents usually follow an observe–reason–act loop: each screen update is converted into a new model decision, and the system issues one concrete GUI action before observing again. V-Droid [5] improves this loop by shifting the LLM

role from action generation to action verification over a discrete action set, reporting 4.3 s per step and 59.5% success on AndroidWorld. VeriSafe [10] similarly focuses on action safety by checking LLM-generated actions against logic constraints derived from the user's goal. These systems improve the cost or correctness of individual steps, but repeated routines still pass through online decision or verification for each interaction. OmniFlow is complementary: a step-wise or verifier-based agent remains the fallback, while matched repeated Functions are served as executable Function cache hits.

Memory-based and programmatic GUI agents. AutoDroid [24], MobileGPT [11], and AppAgent [29] reduce repeated GUI-agent work by building app- or task-specific memory from exploration and prior executions. AndroidWorld's M3A baseline [19] also exposes the benefit of carrying previous actions and reasoning into future attempts. These systems reduce per-goal LLM cost when a warm-start match is found, but their memory is primarily used to guide online reasoning or replay app-specific subtasks. Reuse is limited by per-app cold-start exploration and UI-attribute-based action grounding: XML selectors break on WebViews, custom-rendered feeds, and cross-device layout variation, which forces repair, new exploration, or online LLM calls. ActionEngine [31] and AutoDroid-V2 [25], as well as AgentProg [21], move toward programmatic execution, building app-level state knowledge or generated scripts to reduce online planning. Their reuse unit is a broad app model or complete task program that requires substantial upfront exploration and assumes a stable live app structure. OmniFlow takes a narrower boundary: it treats a verified source run as the asset, caches a Function-level action list, and focuses on transferring each recorded source action to the target GUI instance.

Lightweight and on-device GUI agents. A parallel line of work reduces per-step inference cost by training compact, task-specialized VLMs. UI-TARS [**?**] fine-tunes a vision-language model on large-scale GUI interaction data, improving grounding accuracy and step efficiency while remaining small enough for edge deployment. V-Droid [5] replaces auto-regressive generation with verifier-based action ranking, cutting per-step latency. ShowUI [13] and FocusUI [18] prune visual tokens to reduce inference time. These methods share a common assumption: a model decision is required at every step, and the goal is to make that decision cheaper. OmniFlow addresses a different lever: for recurring GUI workflows, the model call can be eliminated entirely on a cache hit. The two directions are complementary—a lightweight model such as UI-TARS or V-Droid remains the host agent handling cache misses and novel states, while OmniFlow removes model invocations for the repeated Functions that dominate real device workloads.

GUI grounding. Visual grounding models for GUI, such as SeeClick [3], UGround [9], and ShowUI [13], train VLMs to map a natural-language element description to a screen coordinate or bounding box. These models optimize for the *zero-prior* setting: given only a task instruction and the current screenshot, they locate the target element without any recorded demonstration. This makes them general-purpose and immediately applicable to novel apps and screens. OmniFlow addresses the complementary setting: a human demonstration already exists, and the goal is to rebind its recorded actions to the current screen without a model call. GUI grounding models could in principle serve as the transfer oracle in OmniFlow's execution loop, but their per-step inference cost reintroduces the model overhead that OmniFlow is designed to eliminate. Feature-engineered element vectors let Flow transfer recorded targets through structural similarity alone, reserving model calls for genuinely ambiguous or novel states.

Mobile GUI benchmarks and datasets. Benchmarks and datasets such as AndroidWorld [19], Android in the Wild [20], Rico [6], and MobileViews [7] provide the task environments and UI corpora that make mobile GUI automation measurable. They expose the same variation that makes naive replay brittle: dynamic task parameters, changing app state, heterogeneous layouts, and incomplete accessibility annotations. OmniFlow uses these resources as evaluation and analysis substrates rather than as its main method contribution. The method contribution is the record-replay layer that turns a successful mobile execution into a reusable Function and transfers its actions under live validator-scored task instances.

8 Conclusion

We presented OmniFlow, an adaptive replay system for mobile GUI agents that turns past successful executions into reusable `Functions`. The core design separates reusable execution knowledge from live execution control. Omni writes accepted run logs into a Function Store of callable `Functions`, providing a shared tool-like API surface for black-box apps; sparse UDEG nodes serve as the privacy-safe state index for invoking those Functions. Flow applies those Functions on the live device by observing the current state, checking pending actions, transferring recorded targets to current screens, and falling back when the evidence is insufficient.

This representation/execution split changes the cost model of repeated GUI serving. A successful run can write back a reusable `Function`; later compatible users, devices, or task variants can execute the matched Function without re-planning it. Cache hits skip online reasoning for the matched Function, while misses and unsafe transfers return compact page and execution context to the host agent. The online cost is therefore amortized over newly discovered reusable Functions rather than over every user, goal, and repetition.

OmniFlow currently focuses on Android GUI execution and still depends on sufficient state similarity for safe reuse. Large UI redesigns, highly dynamic content regions, and true cold-start goals require fallback to the host agent and later write-back. Its adaptation also has a clear scope: Flow can transfer source actions across current GUI states when structure, geometry, and effect evidence support a confident target, while open semantic decisions can remain VLM, script, or host-agent boundaries. This makes OmniFlow a practical adaptive replay layer: executable, privacy-safe `Functions` are shared, transferred, checked, and refined from future run logs.

A Page-Vector Encoding: Formal Specification

This appendix formalizes the page-vector construction used by the UDEG representation. The purpose of the encoding is to turn a mobile observation into a compact state key that supports fast lookup and action relocation, without storing raw screenshots, raw XML, or source-screen text in the shared payload.

Element vector. Let e_i denote the i-th element in an accessibility tree with n elements. OmniFlow maps e_i to a 64-dimensional vector $\phi(e_i) \in \mathbb{R}^{64}$, formed by concatenating two feature blocks:

$$h(e_i) = [c_i; t_i; a_i; m_i; s_i] \in \mathbb{R}^{16+4+4+4+4},$$
$$g(e_i) = [k_i; b_i; u_i; \ell_i; d_i] \in \mathbb{R}^{8+8+12+2+2}.$$

Here c_i is a text or visual-content hash, t_i is content type, a_i is affordance, m_i is visual prominence, s_i is visual state, k_i is widget class, b_i is boolean attributes, u_i is bounded subtree structure, ℓ_i is hierarchy position, and d_i is the resource-id hint. The final embedding is:

$$\phi(e_i) = [\lambda_h\, \nu(h(e_i))\,;\, \lambda_p\, \nu(g(e_i))], \quad \nu(\mathbf{x}) = \begin{cases} \mathbf{x}/\|\mathbf{x}\|_2, & \|\mathbf{x}\|_2 > 0, \\ \mathbf{0}, & \text{otherwise.} \end{cases}$$

The current implementation uses $\lambda_h = 0.55$ and $\lambda_p = 0.45$.

Human-view block. The human-view block $h(e_i) \in \mathbb{R}^{32}$ encodes rendered-UI cues: a 16-dimensional content hash, 4 content-type dimensions, 4 affordance dimensions, 4 visual-prominence dimensions, and 4 visual-state dimensions.

Programmer-view block. The programmer-view block $g(e_i) \in \mathbb{R}^{32}$ encodes accessibility-XML cues: an 8-dimensional widget-class hash, 8 interaction and state flags, a 12-dimensional bounded-subtree hash, 2 hierarchy dimensions, and 2 resource-id hint dimensions. Raw `text` and `content-desc` strings are not serialized in the minimal shared payload. Local XML-built pages may keep hashed text and selector diagnostics for action mapping, but these fields are opt-in diagnostics rather than the default shared page-vector payload.

Slice pooling. OmniFlow pools element vectors into eight page slices. The current slice order is: R_1 = global text, R_2 = top-bar text, R_3 = footer/navigation text, R_4 = action text, R_5 = state text, R_6 = global UI structure, R_7 = body structure, and R_8 = front-surface layout. The top-bar and footer cuts are quantile-based rather than fixed 10% screen bands, so the same encoder can adapt to pages with different root heights and system UI insets.

Algorithm 2 XML-to-page vector encoding

Require: XML observation X and optional screenshot I
Ensure: normalized page embedding $\mathbf{p}$

$E \leftarrow \textsc{ParseElements}(X)$
for all $e_i \in E$ **do**
  $\mathbf{h}_i \leftarrow \textsc{HumanView}(e_i, I)$
  $\mathbf{g}_i \leftarrow \textsc{ProgrammerView}(e_i, X)$
  $\phi(e_i) \leftarrow [\lambda_h \nu(\mathbf{h}_i); \lambda_p \nu(\mathbf{g}_i)]$
  $\omega_i \leftarrow \textsc{ElementWeight}(e_i, X)$
end for
for $r = 1$ to 8 **do**
  $E_r \leftarrow \{e_i \in E \mid \textsc{Slice}(e_i, X) = r\}$
  if $E_r = \emptyset$ **then**
    $\mathbf{p}_r \leftarrow \mathbf{0}$
  else
    $\bar{\mathbf{p}}_r \leftarrow \frac{\sum_{e_i \in E_r} \omega_i \phi(e_i)}{\sum_{e_i \in E_r} \omega_i}$
    $\mathbf{p}_r \leftarrow \textsc{NormalizeSlice}(\bar{\mathbf{p}}_r)$
  end if
end for
$\mathbf{p} \leftarrow [w_1\mathbf{p}_1; \ldots; w_8\mathbf{p}_8]$
$\mathbf{p} \leftarrow \textsc{NormalizeForLookup}(\mathbf{p})$
return $\mathbf{p}$

For each slice R_r, the page-level vector is pooled with state-aware element weights:

$$\bar{\mathbf{p}}_r = \frac{\sum_{e_i \in E_r} \omega_i \phi(e_i)}{\sum_{e_i \in E_r} \omega_i},$$

where E_r is the set of elements assigned to the slice and ω_i is a positive element weight. If E_r is empty, $\bar{\mathbf{p}}_r = \mathbf{0}$. Each nonzero slice is normalized to $\hat{\mathbf{p}}_r$.

The page-level recall vector is the slice-weighted concatenation:

$$\mathbf{p} = [w_1\hat{\mathbf{p}}_1; \ldots; w_8\hat{\mathbf{p}}_8] \in \mathbb{R}^{512},$$

with fixed slice weights w_r. The vector store normalizes this final vector to unit length for cosine lookup.

Page similarity. The similarity between two pages is computed over normalized page-level vectors:

$$\text{sim}(\mathbf{p}^{(1)}, \mathbf{p}^{(2)}) = \mathbf{p}^{(1)} \cdot \mathbf{p}^{(2)}.$$

This fixed 512-dimensional vector is used for fast candidate recall. The serialized page-vector payload used by action transfer is larger because it also contains element vector records, root bounds, and element bounds. The evaluation therefore reports the actual recalled payload size emitted by the runner instead of assuming a constant page-vector size.

Privacy invariant. The encoding pipeline satisfies the following operational invariant: *the minimal shared page-vector payload contains no raw text or raw content-description string from the source accessibility tree.* Element records retain bounds because action relocation needs geometry. Local diagnostic payloads may additionally include resource IDs, class names, flags, and text/content-description hashes when action mapping needs exact signature evidence. The default shared serialization omits those identity-bearing fields. This is an operational privacy boundary, not a formal differential-privacy claim.

B On-Device Observation Description Cost

Table 6 records the appendix-only diagnostic snapshot used to contextualize on-device memory lookup cost. The snapshot measures the time and token scale for converting one mobile observation into a complete textual description. It is not an end-to-end task result, a success rate, or an energy measurement. Its role is to show why repeated screenshot/XML description is a bottleneck for an on-device replay system, and why OmniFlow uses PageVector lookup as a 0-token state key.

Table 6. Appendix-only single-observation description cost snapshot.

Location	Backend	Input	Tokens	Time
On-device	Llama.cpp + 0.8B-W4 + CPU	screenshot	~2,500	60s+
On-device	MNN-LMM + 0.8B-W4 + CPU	screenshot	~2,500	10s+
On-device	Omniinfer + 0.8B-W4 + NPU	XML	~1k–10k+	3s
Cloud	Qwen-VL-Plus	screenshot	~3,200	3s
OmniFlow	PageVector	XML features	0	4.13 ms/item

References

[1] Reyna Abhyankar, Qi Qi, and Yiying Zhang. 2025. Osworld-human: Benchmarking the efficiency of computer-use agents. *arXiv preprint arXiv:2506.16042* (2025).

[2] Gongwei Chen, Xurui Zhou, Rui Shao, Yibo Lyu, Kaiwen Zhou, Shuai Wang, Wentao Li, Yinchuan Li, Zhongang Qi, and Liqiang Nie. 2025. Less is more: Empowering gui agent with context-aware simplification. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 5901–5911.

[3] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu. 2024. SeeClick: Harnessing GUI Grounding for Advanced Visual GUI Agents. *arXiv preprint arXiv:2401.10935* (2024). https://arxiv.org/abs/2401.10935

[4] Filippos Christianos, Georgios Papoudakis, Thomas Coste, Jianye HAO, Jun Wang, and Kun Shao. 2025. Lightweight Neural App Control. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=BL4WBIfyrz

[5] Gaole Dai, Shiqi Jiang, Ting Cao, Yuanchun Li, Yuqing Yang, Rui Tan, Mo Li, and Lili Qiu. 2025. Advancing mobile gui agents: A verifier-driven approach to practical deployment. *arXiv preprint arXiv:2503.15937* (2025).

[6] Biplab Deka, Zifeng Huang, Chad Franzen, Joshua Hibschman, Daniel Afergan, Yang Li, Jeffrey Nichols, and Ranjitha Kumar. 2017. Rico: A mobile app dataset for building data-driven design applications. In *Proceedings of the 30th annual ACM symposium on user interface software and technology*. 845–854.

[7] Longxi Gao, Li Zhang, Shihe Wang, Pengzhi Gao, Wei Liu, Jian Luan, Shangguang Wang, Yuanchun Li, and Mengwei Xu. 2025. MobileViews: A Million-scale and Diverse Mobile GUI Dataset. *arXiv preprint arXiv:2409.14337* (2025). https://arxiv.org/abs/2409.14337

[8] Lorenzo Gomez, Iulian Neamtiu, Tanzirul Azim, and Todd Millstein. 2013. Reran: Timing-and touch-sensitive record and replay for android. In *2013 35th international conference on software engineering (ICSE)*. IEEE, 72–81.

[9] Boyu Gou, Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. 2025. Navigating the Digital World as Humans Do: Universal Visual Grounding for GUI Agents. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=kxnoqaisCT

[10] Jungjae Lee, Dongjae Lee, Chihun Choi, Youngmin Im, Jaeyoung Wi, Kihong Heo, Sangeun Oh, Sunjae Lee, and Insik Shin. 2025. VeriSafe Agent: Safeguarding Mobile GUI Agent via Logic-based Action Verification. In *Proceedings of the 31st Annual International Conference on Mobile Computing and Networking*. 817–831. doi:10.1145/3680207.3765248

[11] Sunjae Lee, Junyoung Choi, Jungjae Lee, Munim Hasan Wasi, Hojun Choi, Steve Ko, Sangeun Oh, and Insik Shin. 2024. Mobilegpt: Augmenting llm with human-like app memory for mobile task automation. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*. 1119–1133.

[12] Toby Jia-Jun Li, Amos Azaria, and Brad A Myers. 2017. SUGILITE: creating multimodal smartphone automation by demonstration. In *Proceedings of the 2017 CHI conference on human factors in computing systems*. 6038–6049.

[13] Kevin Qinghong Lin, Linjie Li, Difei Gao, Zhengyuan Yang, Shiwei Wu, Zechen Bai, Stan Weixian Lei, Lijuan Wang, and Mike Zheng Shou. 2025. Showui: One vision-language-action model for gui visual agent. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 19498–19508.

[14] Zibin Liu, Cheng Zhang, Xi Zhao, Yunfei Feng, Bingyu Bai, Dahu Feng, Erhu Feng, Yubin Xia, and Haibo Chen. 2025. Beyond Training: Enabling Self-Evolution of Agents with MOBIMEM. *arXiv preprint arXiv:2512.15784* (2025).

[15] Maestro. 2026. Maestro. End-to-End UI Testing for Mobile and Web. https://maestro.dev/.

[16] openatx. 2026. uiautomator2: Android Uiautomator2 Python Wrapper. https://github.com/openatx/uiautomator2.

[17] Antti Oulasvirta, Tye Rattenbury, Lingyi Ma, and Eeva Raita. 2012. Habits make smartphone use more pervasive. *Personal and Ubiquitous computing* 16, 1 (2012), 105–114.

[18] Mingyu Ouyang, Kevin Qinghong Lin, Mike Zheng Shou, and Hwee Tou Ng. 2026. FocusUI: Efficient UI Grounding via Position-Preserving Visual Token Selection. *arXiv preprint arXiv:2601.03928* (2026).

[19] Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William E Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Kenji Toyama, Robert James Berry, Divya Tyamagundlu, Timothy P Lillicrap, and Oriana Riva. 2025. AndroidWorld: A Dynamic Benchmarking Environment for Autonomous Agents. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=il5yUQsrjC

[20] Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap. 2023. Android in the wild: A large-scale dataset for android device control. *arXiv preprint arXiv:2307.10088* (2023).

[21] Shizuo Tian, Hao Wen, Yuxuan Chen, Jiacheng Liu, Shanhui Zhao, Guohong Liu, Ju Ren, Yunxin Liu, and Yuanchun Li. 2025. AgentProg: Empowering Long-Horizon GUI Agents with Program-Guided Context Management. *arXiv preprint arXiv:2512.10371* (2025).

[22] Bryan Wang, Gang Li, and Yang Li. 2023. Enabling conversational interaction with mobile ui using large language models. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*.

1–17.

[23] Yuan Wang, Mingyu Li, and Haibo Chen. 2026. From Imperative to Declarative: Towards LLM-friendly OS Interfaces for Boosted Computer-Use Agents. In *Proceedings of the 21st European Conference on Computer Systems*. 296–310.

[24] Hao Wen, Yuanchun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. 2024. Autodroid: Llm-powered task automation in android. In *Proceedings of the 30th annual international conference on Mobile computing and networking*. 543–557.

[25] Hao Wen, Shizuo Tian, Borislav Pavlov, Wenjie Du, Yixuan Li, Ge Chang, Shanhui Zhao, Jiacheng Liu, Yunxin Liu, Ya-Qin Zhang, et al. 2025. Autodroid-v2: Boosting slm-based gui agents via code generation. In *Proceedings of the 23rd Annual International Conference on Mobile Systems, Applications and Services*. 223–235.

[26] Tingxin Yan, David Chu, Deepak Ganesan, Aman Kansal, and Jie Liu. 2012. Fast app launching for mobile devices using predictive user context. In *Proceedings of the 10th international conference on Mobile systems, applications, and services*. 113–126.

[27] Chaoyun Zhang, Shilin He, Jiaxu Qian, Bowen Li, Liqun Li, Si Qin, Yu Kang, Minghua Ma, Guyue Liu, Qingwei Lin, et al. 2024. Large Language Model-Brained GUI Agents: A Survey. *arXiv preprint arXiv:2411.18279* (2024).

[28] Chaoyun Zhang, He Huang, Chiming Ni, Jian Mu, Si Qin, Shilin He, Lu Wang, Fangkai Yang, Pu Zhao, Chao Du, et al. 2025. Ufo2: The desktop agentos. *arXiv preprint arXiv:2504.14603* (2025).

[29] Chi Zhang, Zhao Yang, Jiaxuan Liu, Yanda Li, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. 2025. AppAgent: Multimodal Agents as Smartphone Users. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. 1–20.

[30] Li Zhang, Longxi Gao, and Mengwei Xu. 2025. Does chain-of-thought reasoning help mobile gui agent? an empirical study. *arXiv preprint arXiv:2503.16788* (2025).

[31] Hongbin Zhong, Fazle Faisal, Luis França, Tanakorn Leesataporn-wongsa, Adriana Szekeres, Kexin Rong, and Suman Nath. 2026. Actionengine: From reactive to programmatic gui agents via state machine memory. *arXiv preprint arXiv:2602.20502* (2026). https://arxiv.org/abs/2602.20502